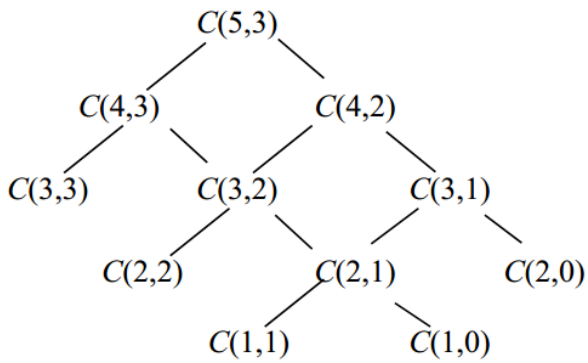
Binomial Coefficient

Write a function that takes 2 parameters N and K and return the value of the binomial coefficient C(N, K).

$$C(n,k) = \begin{cases} 1, & \text{if } k = 0 \text{ or } k = n; \\ C(n-1,k) + C(n-1,k-1), & \text{if } 0 < k < n; \\ 0, & \text{otherwise.} \end{cases}$$

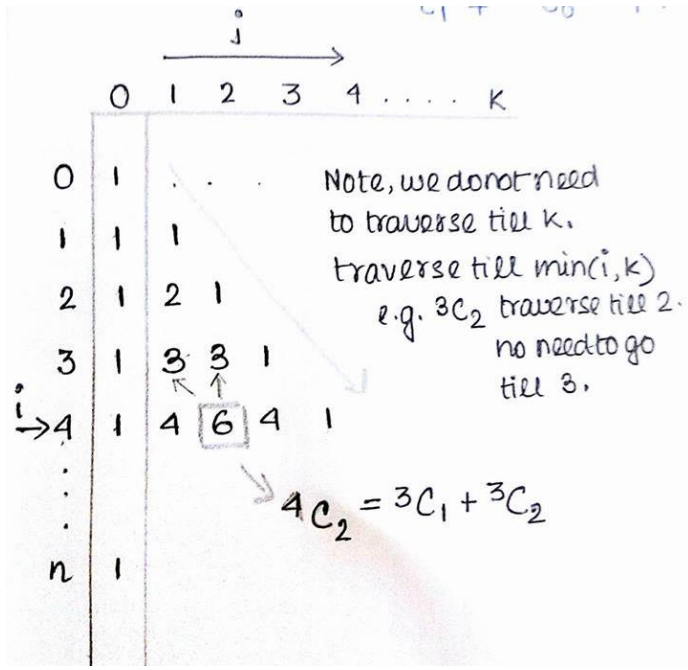
The following "**call tree**" demonstrates repeated (duplicated) computations in a straightforward recursive implementation:




```
int C(int N, int K){
    if(K==0 || K==N)
        return 1;
    else
        return C(N-1, K-1) + C(N-1, K);
}
```

It should be noted that the above function computes the same sub-problems again and again. For the above example C(3,2), C(2,1) are computed twice. Since same sub-problems are called again, this problem has **Overlapping sub-problems property**. So the Binomial Coefficient problem has both properties of DP.

We can avoid calling the same function more than once by constructing a temporary array C[][] in a bottom-up manner.




```
int BC(int N, int K){
    int C[N+1][K+1];
    for(int i=0; i<=N; i++){
        for(int j=0; j<=min(i,K); j++){
            if(j==0 || j==i)
                C[i][j] = 1;
            else
                C[i][j] = C[i-1][j-1] + C[i-1][j];
        }
    }
    return C[N][K];
}
```

Time Complexity: O(N*K)

Auxiliary Space: O(N*K)